

# An Integrated UGV-UAV System for Real-Time Disaster Management: A Multi-Protocol Communication Framework

1<sup>st</sup> Reem Hejazi

*College of Computer and Information Sciences  
Prince Sultan University  
Riyadh, Saudi Arabia  
219410002@psu.edu.sa*

2<sup>nd</sup> Muddesar Iqbal

*College of Communications and Networks Engineering  
Prince Sultan University  
Riyadh, Saudi Arabia  
miqbal@psu.edu.sa*

3<sup>rd</sup> Anees Ara

*College of Computer and Information Sciences  
Prince Sultan University  
Riyadh, Saudi Arabia  
aara@psu.edu.sa*

4<sup>th</sup> Dhafer Jaber

*College of Communications and Networks Engineering  
Prince Sultan University  
Riyadh, Saudi Arabia  
dalmakhles@psu.edu.sa*

**Abstract**—In response to the increasing frequency and severity of natural disasters, the need for efficient and reliable disaster management systems has become more critical than ever. This paper presents the design and implementation of an integrated disaster management framework that leverages Unmanned Ground Vehicles (UGVs) and Unmanned Aerial Vehicles (UAVs) for real-time environmental assessment and search operations. The proposed system employs a multi-protocol communication architecture to enhance data transmission reliability, utilizing the MQTT protocol for lightweight and efficient messaging and RTMP for live video streaming. Additionally, the system incorporates AI-driven face detection and recognition capabilities to improve personnel identification in disaster zones, achieving a mean Average Precision (mAP) of 45.63%. Experimental evaluations demonstrate the effectiveness of the proposed system in maintaining low-latency communication and high-accuracy AI-based recognition, underscoring its potential to enhance disaster response efficiency and decision-making. This research contributes to the advancement of resilient and intelligent disaster management solutions, paving the way for future improvements in autonomous search and rescue operations.

**Index Terms**—Disaster Management, Unmanned Ground Vehicles (UGVs), Unmanned Aerial Vehicles (UAVs), Real-time Video Streaming, MQTT Protocol, RTMP Protocol, Face Detection and Recognition, Communication Protocols, IoT (Internet of Things), Flask Web Server, MongoDB, Nginx, Real-time Data Communication

## I. INTRODUCTION

As the frequency and severity of natural disasters continue to increase, the need for efficient disaster management systems becomes increasingly critical [1]. Recent advances in disaster management have focused on using unmanned vehicle teams to quickly and safely assess disaster-affected areas. These vehicles collaborate with human operators to collect critical environmental data through multiple sensor measurements [2]. In disaster management, unmanned ground vehicles (UGVs)

navigate on the ground, exploring hazardous areas, while unmanned aerial vehicles (UAVs) maintain a fixed position above it [3]. This collaborative robot team enables efficient search and mapping of dangerous environments, with a remote operator monitoring the operation from a safe distance [3].

Despite the significant potential of disaster management systems, there are very few fully integrated solutions available today and none have been widely adopted [4]. This paper presents the design and implementation of a comprehensive disaster management system that integrates multiple communication protocols, middleware, and back-end servers to support UGVs and UAVs. This paper introduces several key contributions to enhance disaster management system capabilities:

- **Multi-protocol Communication** for reliable data transmission and backup in challenging environments.
- **Message Queuing Telemetry Transport (MQTT)** for fast and reliable communication, utilizing a hierarchical network design.
- **Live video streaming** via the Real-Time Messaging Protocol (RTMP).
- **Flexible state machine for UGVs** to handle sensor inputs, control motor movements, and communicate with back-end servers.
- **Face detection and recognition** to enable robots to identify individuals, distinguishing between team members and unidentified persons.

The remainder of this paper is structured as follows: Section 2 reviews related work. Section 3 presents the overall system architecture, including its layers and communication design. Section 4 details the back-end components and their functionalities. Section 5 focuses on real-time data processing and AI technologies. Section 6 discusses the experimental

setup for system evaluation and presents the results. Finally, Section 7 concludes the paper and explores potential future enhancements.

## II. RELATED WORK

Girma et al. [2] propose a framework designed to improve coordination between UAVs, UGVs, and first responders, aiming to enhance the speed and safety of search and rescue operations in disaster environments. The system incorporates autonomous ground-based sensing and navigation using both custom-built and commercial Jackal UGVs, while UAVs, developed with a quadcopter architecture, support aerial sensing, communication, and UGV coordination. A cloud-based remote control station (RCS) is utilized for data collection, real-time visualization, and remote operation, with an antenna tracker extending network coverage. The framework relies on the MQTT protocol for machine-to-machine (M2M) communication.

Hood et al. [3] propose a solution for cooperative exploration in search and rescue operations, utilizing both UGVs and UAVs within a damaged building. The UGV autonomously navigates the environment, while the UAV follows the UGV using vision-based techniques. The system involves a network connecting the UGV, UAV, and Ground Control Station (GCS). Communication between the components is established through an Ethernet connection between the UGV and a portable router, with WiFi used for wireless communication between the GCS and the router.

Surmann et al. [5] analyze the insights gained from the deployment of ground and aerial robots in disaster response scenarios under the German Rescue Robotics Center (DRZ) task force. The system includes a specialized emergency vehicle designed to transport multiple UAVs and a mid-sized UGV while establishing a mobile communication hub. It houses two operator workstations embedded within its own network and server infrastructure, providing centralized robot control and data management. The DRZ task force also develops SKATES, a resilient communication platform that integrates multiple network technologies such as WiFi, 4G, 5G, and IP-Mesh to ensure uninterrupted connectivity for robot operations and data transmission even in challenging environments.

Aretoulaki et al. [6] introduce a drone-enabled humanitarian logistics platform called "AIRDROP," designed to assist first responders and decision-makers throughout the various stages of a disaster. The platform leverages unmanned vehicles, including UAVs, UGVs, and USVs, to provide a diverse array of logistics services. The system architecture consists of several key components, including a Command Center Web Application (CCWA) for mission management, a First Responder Mobile Application (FRMA) for field operations, a Mission Planning System (MPS) for trajectory deconfliction, and a Decision Support System (DSS) for integrating and exchanging data between all components. The communication network relies on UAVs acting as relays to extend coverage, using Wi-Fi, Bluetooth, and cloud connectivity for seamless data sharing.

Despite advances, common challenges persist, including network reliability, data integration, and seamless operations. Our work addresses these gaps by improving the reliability, efficiency of communication, and real-time capabilities of the system. We employ a multi-protocol framework with backup strategies, a scalable hierarchical MQTT network for data integrity, and AI-driven face detection to improve decision-making.

## III. SYSTEM ARCHITECTURE

### A. Layered System Design

The system is structured into four primary layers:

- **Hardware Layer:** Comprises various robotic units, including UGVs (e.g., Smart Tank Robot, Jetbot AI Robot Car), and UAVs (drones). Additional equipment includes charging stations and signal extenders.
- **Middleware Layer:** Facilitates communication using local MQTT brokers within each robot team for real-time data exchange and offline operation. A LoRa gateway enables UAV communication and an access point connects UGVs at the gateway station.
- **Cloud Layer:** Hosts essential back-end components for system operations, ensuring data storage, processing, and accessibility.
- **Application Layer:** Provides a mobile app interface for users to monitor and control robots remotely.

### B. Multi-protocol Communication Framework

To ensure robust and adaptable communication, our framework integrates multiple protocols across different network layers, as illustrated in Fig. 1. All physical-layer robots communicate with the cloud back-end through the local MQTT broker. The mobile app can control UGVs via the cloud, while UAVs are controlled through a drone controller. The system leverages three data link layer protocols that are used across all robots: WiFi, LoRa, and 5G. WiFi is the primary protocol due to its speed and reliability, while LoRa serves as a secondary option when WiFi is unavailable. Both WiFi and LoRa are used to communicate with the local broker. 5G is the backup protocol that allows the robot to communicate directly with the cloud. All robots are also equipped with GPS modules for location tracking. For application protocols, MQTT, HTTP, RTMP, and MAVLink are employed. Robots communicate with the broker using MQTT and the broker communicates with the cloud using the same protocol. For specific RESTful APIs, robots communicate directly with the cloud via HTTP, and for live video streaming, RTMP is used. The mobile app utilizes all three protocols (HTTP, MQTT, RTMP) to interact with the cloud. Lastly, MAVLink, a specialized protocol for drone communication, is used in the drone controller. By integrating multiple protocols, our framework improves network resilience, scalability, and operational flexibility, allowing robots to maintain connectivity and adapt to changing network conditions in dynamic environments.

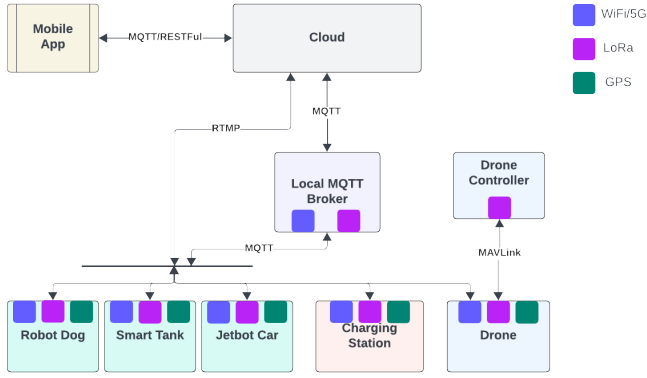


Fig. 1. Middleware Design

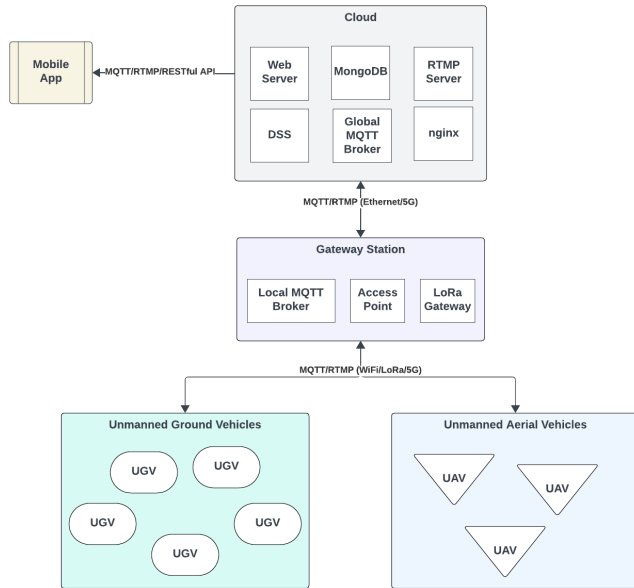


Fig. 2. System Design

#### IV. PROPOSED BACKEND DESIGN

In the proposed system illustrated in Fig.2, all UGVs and UAVs communicate with the back-end in the cloud through the gateway station, which contains an access point for Wi-Fi connection, an MQTT broker for MQTT messages, and a LoRa gateway for UAV communication. Then, mobile users can communicate with the cloud to access the system's capabilities. The backend consists of six components:

##### A. Web Server

The Flask [7] server facilitates communication between the mobile app and robots via RESTful APIs. It follows the

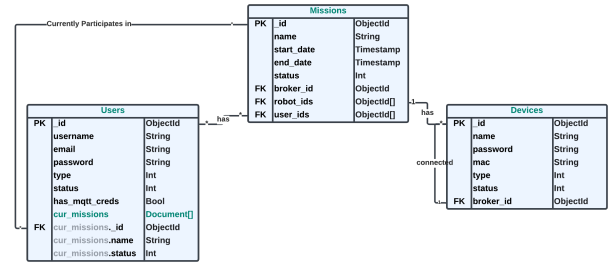


Fig. 3. Database Design

Model-View-Controller (MVC) architecture, a design pattern that separates concerns: models manage data, views handle user interactions (in this case, mobile app interfaces), and controllers process logic and coordinate between models and views. The server returns JSON-based responses and implements token-based authentication for regular and admin users. A validation class ensures input integrity, and an MQTT interface manages admin-specific messages.

##### B. Database Server

MongoDB [8], a scalable NoSQL database, is used for efficient IoT data storage. The database consists of three collections as shown in Fig. 3: Users collection stores profile details, including credentials (ID, username, email, password), status, type (regular/admin), and an MQTT account flag. Missions associated with a specific user are embedded within the user document for quick access, with only ID, name, and status. Devices collection maintains records of devices, including credentials, MAC address, status, type (UGVs, UAVs, Brokers), and local broker ID. Missions collection tracks task-specific user and device assignments, with fields such as ID, name, status, start/end dates, local broker ID, and participating robot/user IDs.

##### C. Nginx

An open-source web server software that also functions as a reverse proxy server, load balancer, and HTTP cache [9]. In our case, it works as a reverse proxy for the web server.

##### D. RTMP Server

This server enables live video streaming, and all robots can connect to it using frameworks such as FFmpeg [10]. The mobile app accesses the streams via an RTMP player by connecting to the same RTMP link. For the deployment of the RTMP server, Nginx [9] is used, as it includes a module for the streaming of RTMP. Nginx [9] is configured to function as an RTMP server with optimized parameters to minimize latency, ensuring real-time video streaming for efficient monitoring through the mobile app.

##### E. Decision Support System (DSS)

An information system that supports decision-making activities for robots. It has functionalities like predicting disasters, suggesting recommendations, deciding the best actions for

robots, sending alerts and notifications to users, and sending commands to robots.

#### F. Global MQTT Broker

MQTT brokers follow a hierarchical structure, with one global broker deployed in the cloud and multiple local brokers connected to it, each located at disaster sites. The global broker manages communication between the local MQTT brokers, including sensor data and control commands. MQTT is used for faster and easier communication, allowing the mobile app to connect directly to the global broker to publish/subscribe to the robots. The Mosquitto software [11] is used for global and local brokers, configured with appropriate access control and settings.

Each mission requires one local broker. A Raspberry Pi 4 [12] is used to set up and deploy the local MQTT broker, ensuring proper configuration, authentication, and access control. This local broker is essential for enabling communication between on-site robots and connecting them to the cloud broker. In addition to the MQTT broker program, another program runs on the Raspberry Pi: the bridge program, developed in Python, which facilitates communication between the local and global brokers. The bridge program also implements queuing and file buffering mechanisms to ensure effective data buffering and integrity.

#### V. REAL-TIME DATA PROCESSING AND AI INTEGRATION

The data collected from UGV and UAV sensors is transmitted to local brokers by MQTT. These local brokers then relay the data to the global broker using a bridging program. All incoming data are integrated into the cloud through the Decision Support System (DSS). In addition, video streams undergo preprocessing by AI algorithms, including face detection and recognition, before being transmitted. All data communication is managed within the UGV using a state machine, ensuring efficient processing and transmission.

##### A. Face Detection & Recognition

Face detection and recognition are implemented in disaster management UGVs to enhance their ability to identify individuals within a team in real-time. YOLOv8 [13], a fast and accurate deep learning model, is used for object detection. A pre-trained version is integrated into the robot's program for face detection. After capturing a frame from the robot's camera, the model processes it and returns the coordinates of detected human faces. Bounding boxes are then drawn around these faces and the processed frame is sent to the RTMP server, allowing mobile users to access the stream.

For face recognition, the FaceNet algorithm [14] is selected due to its high accuracy. Facial images of team members were collected and encoded into embeddings, which were stored on the cloud using a Flask server. The robot retrieves these embeddings and compares them with newly detected faces to determine identity. If a match is found, the corresponding name is displayed; otherwise, "Unknown" is printed. This process occurs after face detection and before transmitting

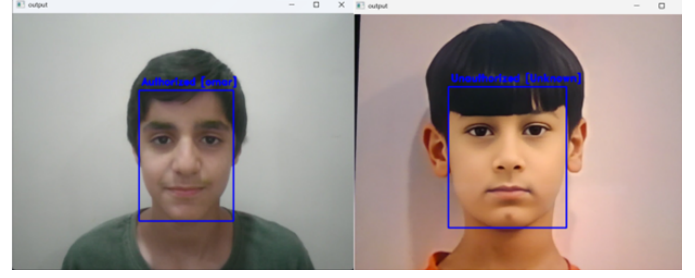


Fig. 4. Face Detection & Recognition Output

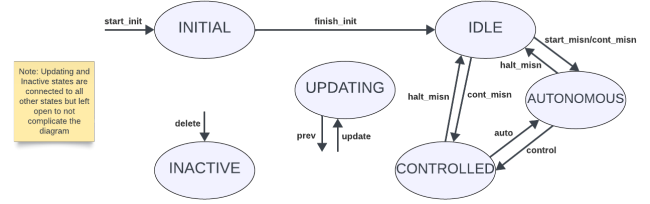


Fig. 5. UGV State Machine

the video feed, ensuring reliable identification in disaster scenarios. The results are shown in Fig. 4.

##### B. Unmanned Ground Vehicle Program

The system features a complex backend with numerous components, which can present integration challenges. To address this, a state machine was designed for the UGV with the following states, as shown in Fig. 5:

- **Initial:** This state handles all configuration and initialization tasks.
- **Idle:** The robot is in this state when it is inactive and awaiting commands from administrators.
- **Autonomous:** The robot operates independently in this state, following pre-programmed behaviors.
- **Controlled:** In this state, the robot is manually controlled by the users through a mobile app.
- **Updating:** The robot is updating its name or password in this state.
- **Inactive:** The robot enters this state when it is deactivated.

Additionally, an MQTT interface is integrated to connect the UGV to the local MQTT broker, operating on a separate thread to avoid blocking the state machine. Furthermore, an RTMP interface is incorporated to enable the UGV to stream live video from its camera, utilizing OpenCV [15] and FFmpeg [10]. This video streaming interface operates in a separate process to ensure smooth performance.

#### VI. EXPERIMENT & RESULTS

##### A. Experimental Setup

To evaluate the efficiency of the system, we measured the round-trip latency under various scenarios. The MQTT messages, local and global, were transmitted 100 times with

a size of 1KB, and the average latency was calculated for all transmissions. Similarly, the web server was pinged 100 times, and the average latency was computed. For video streams, we calculated the average difference between the timestamps of 100 frames, comparing when they were sent and when they were received. In addition, the mean precision of the face recognition and detection algorithms was evaluated.

The hardware specifications for the various components of the system are provided in Table I.

### B. Results & Discussion

The latency results are presented in Table II. For face detection, we used the pre-trained YOLOv8 model 'YOLOv8n-Face', which has a mean average precision (mAP) of 37.5% [16]. For face recognition, we ran the algorithm on the Wild Face Recognition dataset [17], consisting of 1680 individuals after data cleaning. A subset of data 10% was used for testing, and the algorithm achieved an mAP of 45.63%.

Finally, a comparison highlighting the key features of our proposed system in relation to other existing systems is shown in Table III. Our system supports multi-protocol communication, video streaming, MQTT communication, and AI/ML algorithm integration.

TABLE I  
HARDWARE SPECIFICATIONS OF SYSTEM COMPONENTS

| Component     | Hardware Platform      | Specification                                                                         |
|---------------|------------------------|---------------------------------------------------------------------------------------|
| Cloud Servers | Digital Ocean Droplet  | Intel Xeon Platinum 8168 CPU<br>8GB RAM<br>50GB Disk<br>Ubuntu 24.04 (LTS) x64        |
| Local Broker  | Raspberry Pi 4 Model B | Quad Core Cortex-A72 (ARM v8)<br>8GB RAM<br>IEEE 802.11ac wireless<br>Raspberry Pi OS |
| Dummy UGV     | Raspberry Pi 4 Model B | Quad Core Cortex-A72 (ARM v8)<br>8GB RAM<br>IEEE 802.11ac wireless<br>Raspberry Pi OS |
| Mobile App    | Samsung Galaxy A70     | Snapdragon 675 Octa-core<br>8GB RAM<br>Android 11                                     |

TABLE II  
LATENCY RESULTS FOR DIFFERENT SCENARIOS

| Case                      | Average Latency (ms) |
|---------------------------|----------------------|
| MQTT Local Communication  | 45.00                |
| MQTT Global Communication | 200.78               |
| Web Server Responses      | 214.00               |
| Video Streams             | 666.85               |

## VII. CONCLUSION & FUTURE WORK

In this paper, we presented the design and implementation of a comprehensive disaster management and recovery system that uses UGV and UAV. The system is equipped with a robust communication architecture, integrating multiple protocols

TABLE III  
FEATURE COMPARISON

| Feature                      | [2] | [3] | [5] | [6] | Our Proposed System |
|------------------------------|-----|-----|-----|-----|---------------------|
| Multi-protocol Communication |     |     | ✓   |     | ✓                   |
| Video Streaming              |     | ✓   | ✓   | ✓   | ✓                   |
| MQTT Communication           | ✓   |     |     |     | ✓                   |
| AI/ML Algorithms             |     |     | ✓   | ✓   | ✓                   |

such as MQTT, RTMP, and HTTP, as well as essential back-end components such as Flask [7], MongoDB [8], and Nginx [9]. Key features include live video streaming, real-time data communication, a flexible state machine for UGVs, and face detection and recognition to enhance situational awareness. For future work, we plan to enhance the system's scalability and reliability by optimizing the communication protocols and introducing machine learning-based decision-making capabilities for the robots. Furthermore, the system could be extended to include more advanced disaster prediction algorithms, and further integration with other types of unmanned vehicles could improve the overall operational efficiency of the disaster management process. All code associated with this research is publicly available in the author's GitHub repository [18].

## REFERENCES

- [1] M. Krichen, M. S. Abdalzaher, M. Elwekeil, and M. M. Fouda, "Managing natural disasters: An analysis of technological advancements, opportunities, and challenges," *Internet of Things and Cyber-Physical Systems*, vol. 4, pp. 99–109, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2667345223000500>
- [2] A. Girma, N. Bahadori, M. Sarkar, T. G. Tadewos, M. R. Behnia, M. N. Mahmoud, A. Karimoddini, and A. Homaifar, "IoT-enabled autonomous system collaboration for disaster-area management," *IEEE/CAA Journal of Automatica Sinica*, vol. 7, no. 5, pp. 1249–1262, 2020.
- [3] S. Hood, K. Benson, P. Hamod, D. Madison, J. M. O'Kane, and I. Rekleitis, "Bird's eye view: Cooperative exploration by ugv and uav," in *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, 2017, pp. 247–255.
- [4] M. Careem, C. De Silva, R. De Silva, L. Raschid, and S. Weerawarana, "Sahana: Overview of a disaster management system," in *2006 International Conference on Information and Automation*, 2006, pp. 361–366.
- [5] H. Surmann, K. Daun, M. Schnaubelt, O. von Stryk, M. Patchou, S. Böcker, C. Wietfeld, J. Quenzel, D. Schleich, S. Behnke, R. Gräfe, N. Heidemann, D. Slomma, and I. Kruijff-Korbayová, "Lessons from robot-assisted disaster response deployments by the german rescue robotics center task force," *Journal of Field Robotics*, vol. 41, no. 3, pp. 782–797, 2024. [Online]. Available: <https://onlinelibrary.wiley.com/doi/abs/10.1002/rob.22275>
- [6] E. Aretoulaki, S. Ponis, G. Plakas, D. Tzanetou, and A. Kitsantas, "A proposed drone-enabled platform for holistic disaster management," in *International Conferences on Applied Computing 2023 and WWW/Internet 2023*, 10 2023.
- [7] P. Projects, "Flask: Web development, one drop at a time," <https://flask.palletsprojects.com/>, 2010.
- [8] M. Inc., *MongoDB: The Developer Data Platform*, <https://www.mongodb.com/>, 2009.
- [9] I. Sysoev and I. Nginx, "Nginx: High performance load balancer, web server, and reverse proxy," <https://nginx.org/>, 2004.
- [10] F. Developers, "Ffmpeg," <https://ffmpeg.org/>, 2000.
- [11] E. Foundation, "Mosquitto: An open source mqtt broker," <https://mosquitto.org/>, 2020.
- [12] R. P. Foundation, "Raspberry pi: Single-board computers for education and innovation," <https://www.raspberrypi.org/>, 2020.

- [13] Ultralytics, “Explore ultralytics yolov8,” <https://github.com/ultralytics/ultralytics/blob/main/docs/en/models/yolov8.md>, 2023.
- [14] Davidsandberg, “Face recognition using tensorflow,” <https://github.com/davidsandberg/facenet>, 2018.
- [15] O. Contributors, “Opencv: Open source computer vision library,” <https://opencv.org/>, 2000.
- [16] Lindevs, “Yolov8-face: Face detection with yolov8,” <https://github.com/lindevs/yolov8-face>, 2024.
- [17] A. A. Jha, “Lfw people (face recognition),” <https://www.kaggle.com/datasets/atulanandjha/lfwpeople>, 2020.
- [18] R. Hejazi, “Disaster management system backend,” <https://github.com/rem2718/disaster-management-backend>, 2024.